

BSS TIME COLLECTION

Project Context, Oracle Integration Baseline
and Engineering Handover for Codex

Working document - MVP v0.1

Baseline date: 9 October 2026

[GitHub: kaciker/BSS_TimeCollection](#)

Purpose of this document

Give Codex enough business, Oracle, functional, technical and deployment context to evolve the application without turning it into a second Time & Attendance system or breaking the audit and delivery guarantees already agreed.

1. Executive summary

BSS Time Collection is an External Time Collection Device (TCD) application for factory use. Its job is to capture attendance/time events from a badge/RFID reader or on-screen/manual identifier, persist every event internally, keep a complete technical delivery history, and reliably deliver the event to Oracle HCM Time and Labor.

Golden design rule

BSS Time Collection records and delivers facts. Oracle HCM Time and Labor interprets those facts.

The application is intentionally not a full T&A product. Oracle remains responsible for employee resolution, schedules, IN/OUT matching, business validation, exceptions, time cards, working-time calculations and payroll-related processing. The BSS application adds the factory capture layer that Oracle explicitly supports as a third-party time collection device.

Current engineering baseline

Item	Baseline
Repository	https://github.com/kaciker/BSS_TimeCollection
Branch / baseline commit	main / e522240 (baseline at document creation)
Deployment host	Docker-Main - 192.168.31.156
Deployment path	/opt/BSS_TimeCollection
Published port	8092
Stack	React + Nginx / FastAPI / PostgreSQL / Python delivery worker
Oracle delivery	Disabled until the real tenant, authentication and mappings are validated
Application authentication	Not implemented by design; final identity boundary is pending

2. Why the project exists

Business trigger. During the scope discussion, the Fizz commercial team clarified that Oracle HCM for Time & Attendance is the SaaS processing layer, but that the factory tracking/capture layer must also exist. Their message was effectively a scope warning: Hutchinson appeared to expect both the Time & Attendance solution and the factory time-tracking system, while Fizz had understood that the data/capture infrastructure already existed.

Fizz scope clarification (project input)

"Oracle HCM for TT&A is a SaaS tool and not a Full Time and Attendance Solution with tracking system in factory and so on... We need to realigned with the scope if Hutchinson expect time tracking solution + time tracking system."

Oracle documentation confirms the same architectural split. Oracle Time and Labor accepts events from third-party time collection devices such as badge or biometric readers. Imported device events are validated, mapped and then used by Oracle to create or complete time entries. This is exactly the gap BSS Time Collection is intended to fill.

Business objective

- Provide a simple, robust factory terminal experience based on screen + RFID/badge reader, without requiring a proprietary full T&A terminal ecosystem.
- Allow the same software to run on different compatible hardware and support many terminals across a plant.

- Capture the event even when Oracle is temporarily unavailable, so a SaaS/network outage does not prevent a worker from clocking.
- Keep BSS-owned audit evidence of what was captured, when, where and how it was delivered.
- Integrate with Oracle using Oracle-supported Time Event interfaces and mappings instead of reimplementing Oracle business logic.

3. Scope boundary: what the application must and must not do

Owned by BSS Time Collection

- Terminal identity and per-terminal configuration.
- RFID/badge capture and manual/keypad identification. RFID hardware integration remains pluggable.
- Configurable actions per terminal, including the visible labels and supplier event codes sent to Oracle.
- Precise event timestamp with timezone/offset.
- Minimal derived INSIDE/OUTSIDE state only to simplify the terminal UX.
- Durable internal event storage before any external delivery attempt.
- Asynchronous Oracle delivery, retries for clearly transient failures, and reconciliation-safe handling of ambiguous timeouts.
- Separate technical transmission history and operational logs.
- Admin/configuration capability and health/operability endpoints.

Explicitly out of scope

Do not implement	Why
Employee master data / names	Oracle owns employee identity and employment context.
Shift and schedule management	Oracle Time and Labor owns schedule-aware processing.
Worked-hours calculation / overtime	Business calculation belongs to Oracle rules.
IN/OUT functional pairing	Oracle matches device In and Out events and creates/completes time entries.
Missing-punch correction	Oracle exception/rule processing owns functional correction workflows.
Absence, approval, payroll and time cards	These are Oracle T&A / HCM responsibilities.
Functional validation of the worker event	BSS validates technical shape and local configuration only; Oracle validates business meaning.

Codex guardrail

Do not add T&A business features simply because they appear useful. If a proposed feature needs worker schedules, payroll rules, worked-hour calculations, absence logic or functional punch correction, stop and treat it as a scope decision, not an implementation detail.

4. Oracle integration model and standards

Oracle model. A third-party time collection device transfers time data through the Time Event REST service. Oracle states that correctly formed resources are stored for further functional validations, while inaccurately formed resources are returned in error. Oracle then applies device event mappings, device rules and matching logic.

Core REST resource

POST /hcmRestApi/resources/11.13.18.05/timeEventRequests

The Oracle resource can contain one or more time events. The current BSS implementation deliberately sends one local event per request to maximize traceability and make reconciliation easier. Batching can be added later if a measured throughput requirement justifies it.

Canonical fields to preserve

Oracle field	Meaning	BSS handling
requestNumber	Unique request identifier from the device/supplier	Generate uniquely per delivery request and retain locally.
sourceId	Oracle source identifier	Configurable; do not hard-code tenant-specific assumptions.
requestTimestamp	Time the delivery request is produced	Generate at send time.
deviceId	External device/terminal identifier	Snapshot from terminal configuration at capture time.
eventDateTime	Actual time of the worker event	Captured once, timezone-aware, immutable.
supplierDeviceEvent	Event code understood by the device supplier and mapped in Oracle	Configured action code; snapshot on the event.
reporterId	Worker/badge/person identifier	Raw identifier captured by RFID or keypad.
reporterIdType	Identifier type such as BADGE or PERSON	Configurable; BADGE is the expected factory direction unless Oracle implementation decides otherwise.
timeEventAttributes[]	Optional time attributes	Action-level optional JSON only when required by Oracle mapping.

Oracle event semantics

Oracle maps supplier device events to four application-event patterns: **In, Out, In and Out, Out and In**. This means BSS can expose business-friendly buttons such as Start work, End work, Break, Meal, Medical or No work while Oracle decides how those supplier codes map into Oracle Time and Labor events and attributes.

Important processing boundary

Transport acceptance is not business acceptance

A successful HTTP response and returned Oracle IDs prove that Oracle accepted/stored the REST resource at the transport/repository level. Oracle still performs downstream functional validation, event mapping, matching and exception processing. BSS must never treat SENT as proof that the time event is functionally valid for payroll/time-card purposes.

Oracle response data

Oracle examples return **timeEventRequestId** and per-event **timeEventId**. Store these identifiers with the local event whenever available so BSS can correlate its immutable event with the Oracle-side resource.

5. Terminal UX and minimal worker state

Key UX requirement. The worker should not have to choose between IN and OUT every time the badge is presented. The application therefore derives a minimal interaction state from the latest relevant event already stored by BSS.

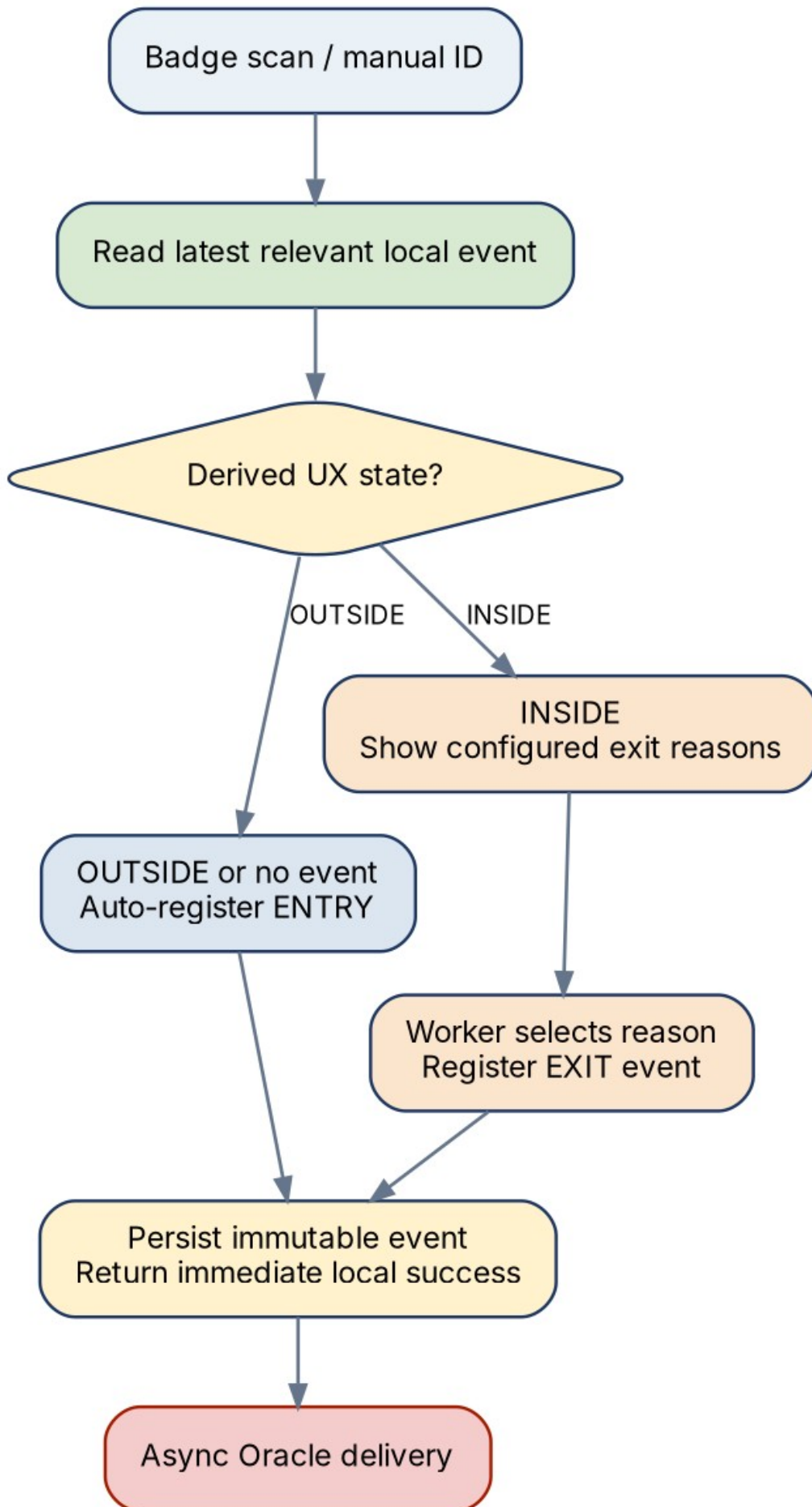


Figure 1 - Derived interaction state. This is a UX convenience, not authoritative T&A state.

State rule

```
last_state_effect = ENTER  -> INSIDE
last_state_effect = EXIT   -> OUTSIDE
no_prior_event      -> OUTSIDE
```

- If OUTSIDE: badge presentation automatically records the configured entry action. No IN/OUT choice is shown.
- If INSIDE: badge presentation shows only the configured exit actions/reasons. The worker selects the reason and the app records the exit event.
- The state is derived from BSS events, not queried from Oracle during the interaction. This keeps the terminal fast and tolerant of Oracle/network outages.
- All terminals must share the same backend state. The state must never live only in a single physical terminal because a worker may present the badge at a different terminal.
- If the local event sequence is ambiguous or a future use case allows transfers between work cells without an OUT/IN cycle, do not invent behavior. Add an explicit product decision or a transfer semantic.

Critical distinction

BSS may know that its latest captured event implies INSIDE for UI purposes. That does not make BSS the legal or functional authority for attendance. Oracle remains the functional system of record and may contain corrections or exceptions unknown to the terminal.

6. Internal persistence, auditability and logs

Project decision. Every captured event must be kept in the BSS database even though Oracle is the functional T&A system. The technical delivery history must also be retained separately. This is deliberate: it provides local evidence of what the factory terminal actually captured and a trace of every integration attempt.

Event facts vs. delivery attempts

Immutable event fact	Technical transmission attempt
event_id UUID	attempt number / timestamp
terminal_id + device_id snapshot	target endpoint
reporter_id + reporter_id_type	HTTP status / result
identification_method: RFID or KEYPAD	duration / timeout information
internal action code/label snapshot	error code / error message
supplier_device_event snapshot	Oracle response snapshot where appropriate
event_datetime + created_at	correlated local event_id
delivery_status and Oracle IDs	no rewriting of the original event fact

Required persistence behavior

- Persist locally before showing success to the worker.
- Never change the captured event timestamp because Oracle was unavailable or delivery was delayed.
- Keep terminal/device and supplier-event snapshots on the event so later configuration changes do not rewrite history.
- Prefer append-only event facts. Corrections, if ever required, must be explicit new records or audited administrative actions rather than silent edits.
- Keep technical logs/attempts separate from the event row to avoid duplicating or mutating the business fact.
- Retention duration and purge policy are not yet defined; do not introduce destructive cleanup without an explicit decision.

7. Technical architecture



Figure 2 - Current modular architecture and Oracle delivery boundary.

Current components

Component	Technology	Responsibility
Web terminal / admin	React + TypeScript + Vite, served by Nginx	Kiosk interaction, manual ID today, admin/configuration surfaces.
API	FastAPI + Pydantic settings	Terminal/action configuration, event capture, derived UX state and admin APIs.
Persistence	PostgreSQL + SQLAlchemy + Alembic	Immutable event facts, config and technical attempt history.
Delivery worker	Separate Python process + httpx	Claims pending events and delivers them to Oracle independently of the terminal request.
Deployment	Docker Compose	Single reproducible stack with web, API, worker and DB services.
Tests/CI	pytest + frontend build + Compose smoke test	Protect backend, frontend and end-to-end deployment behavior.

Engineering structure

The project must remain modular. Keep domain/event logic, persistence, HTTP routes, integration adapters, worker orchestration and frontend components separated. **Do not collapse the application into one large file or a small set of catch-all modules.** New work should extend the existing boundaries and refactor when a module starts accumulating unrelated responsibilities.

8. Oracle delivery, retries and reconciliation

Current delivery state model

Status	Meaning
PENDING	Captured locally and waiting for delivery.
SENDING	Claimed by the worker and currently being processed.
RETRY	A clearly transient, non-ambiguous failure occurred; safe to retry.
SENT	Oracle accepted the HTTP request; Oracle business validation may still occur later.
REJECTED	Non-retryable response or payload/configuration problem requiring investigation.
UNKNOWN	Delivery outcome is ambiguous, typically after a timeout where Oracle may already have persisted the event. Do not blindly replay.

Why UNKNOWN exists

Blind retries can duplicate a punch if Oracle persisted the event but the client lost the response. Until the real Oracle tenant behavior and any idempotency/reconciliation mechanism are validated, an ambiguous timeout

must be quarantined as UNKNOWN rather than automatically replayed. This is a safety decision in the BSS adapter, not an Oracle guarantee.

Timestamp rules

- eventDateTime = the real capture time of the worker event, with timezone/offset.
- created_at = when BSS persisted the event.
- requestTimestamp = when the Oracle delivery request is constructed.
- sent_at = when BSS receives the successful Oracle transport response.
- A delayed send must never replace eventDateTime with the later delivery time.

9. Configuration and authentication boundary

Terminal configuration

- Stable terminal identity and Oracle-facing deviceId.
- Identification mode per terminal: RFID, KEYPAD or BOTH.
- Enabled entry/exit actions per terminal and their supplierDeviceEvent codes.
- Optional action-level Oracle timeEventAttributes.
- Active/inactive state and future health/last-seen metadata as required.

Authentication decision

Version 0.1 intentionally has no application authentication. The final model is pending because the module may be embedded inside another authenticated application or may receive its own authentication/gateway layer. Codex must keep authentication outside the core event/domain services so the choice can be changed later without rewriting capture or Oracle delivery logic.

Security warning for the current home deployment

The current instance is for development/trial use on a trusted home network. Do not expose the application directly to the Internet while authentication is absent. Oracle credentials must never be committed to Git and Oracle delivery stays disabled until a controlled tenant configuration is available.

10. Current deployment baseline

GitHub

https://github.com/kaciker/BSS_TimeCollection

Baseline commit at the time of this document: **e522240**. The latest CI run for that baseline completed backend tests, frontend build and a Docker Compose smoke flow successfully.

Docker-Main development deployment

Parameter	Value
Host	Docker-Main
IP	192.168.31.156
Path	/opt/BSS_TimeCollection
Published web port	8092
Terminal demo	http://192.168.31.156:8092/terminal/DEMO-01
Admin	http://192.168.31.156:8092/admin
Health	http://192.168.31.156:8092/api/v1/health/ready
Health response	{"status":"ready"}

Parameter	Value
Oracle delivery	ORACLE_ENABLED=false
Authentication	None

Compose services

- db - PostgreSQL persistent event/configuration store.
- api - FastAPI application; runs database migrations before serving.
- worker - independent Oracle delivery loop.
- web - Nginx serving the React application and proxying /api to FastAPI.

Safe update workflow for Codex

1. Inspect current repository state and pull latest main before changing anything.
2. Make changes in coherent modules; preserve existing event/audit behavior unless the requirement explicitly changes it.
3. Run backend tests and frontend build.
4. Validate docker compose config and perform a smoke deployment.
5. For the home server, update from /opt/BSS_TimeCollection using docker compose up -d --build after pulling the intended commit.
6. Verify /api/v1/health/ready and the terminal flow before considering the block closed.
7. Do not enable Oracle in the home environment unless an explicit controlled test endpoint/tenant is provided.

11. Working contract for Codex

Expected behavior

Treat this document as the product/architecture baseline, but always inspect the live repository before editing. Complete engineering blocks end-to-end: inspect, implement, test, build, deploy when appropriate, validate and report exact results. Stop only for a real blocker, destructive action, security risk or unresolved functional decision.

Non-negotiable guardrails

- Do not turn BSS Time Collection into a parallel Oracle Time & Attendance engine.
- Do not delete or silently mutate historical time events to simplify implementation.
- Do not merge technical delivery logs into the event fact in a way that loses retry history.
- Do not make Oracle availability part of the worker-facing success path. Local persistence must happen first.
- Do not use Oracle as an online state lookup dependency for each badge scan.
- Do not infer employee identity/name/schedule unless a new Oracle-to-device synchronization requirement is explicitly approved.
- Do not hard-code tenant URL, authentication, sourcelid, device mappings or supplier event codes.
- Do not add authentication dependencies inside domain logic. Keep the future auth layer replaceable.
- Keep code modular and maintainable; split responsibilities before files/classes become catch-all components.
- Preserve Docker Compose as the standard deployment path.

Definition of done for a change block

Area	Minimum completion evidence
Functional	Requirement demonstrated through terminal/admin/API behavior.
Persistence	No loss of event facts; migrations included and safe.
Oracle adapter	Payload/response behavior covered by tests or mocks; no unvalidated tenant assumptions.

Area	Minimum completion evidence
Backend	Relevant pytest suite passes.
Frontend	TypeScript/Vite build passes; interaction tested.
Compose	docker compose config valid; stack builds/starts.
Runtime	Health endpoint ready and key smoke flow succeeds.
Documentation	README/docs updated when architecture, contract or deployment behavior changes.

12. Open decisions - do not silently resolve

- Physical RFID reader model, driver/interface and whether it behaves as keyboard wedge, serial/USB device or browser-accessible local service.
- Final application authentication model: embedded/gateway identity vs. standalone authentication.
- Real Oracle tenant base URL and network path/proxy requirements.
- Oracle authentication mechanism and service account/credential ownership.
- Final sourceId, reporterIdType, supplier code and supplierDeviceEvent naming/mappings configured by the Oracle implementation team.
- Whether Oracle timeEventAttributes are needed for each event type and their exact values.
- Reconciliation/idempotency strategy against the real Oracle tenant for ambiguous timeouts.
- Retention period and purge/archive policy for BSS events and technical logs.
- Whether worker transfers between cells/terminals require an explicit transfer event or remain simple OUT/IN interactions.
- Whether Oracle-to-device exports (person/employment/schedule data) are ever required. They are intentionally not part of MVP.

13. Official Oracle reference set

Use these Oracle sources as the primary external standard when changing the integration. They were re-checked against Oracle documentation on 9 October 2026.

[01] Overview of Time Collection Device and Web Clock Event Processing - Defines the third-party time-device processing flow, validation and creation/completion of time entries.
<https://docs.oracle.com/en/cloud/saas/human-resources/faitl/overview-of-time-collection-device-and-web-clock-event.html>

[02] How You Import Data from Time Collection Devices - Confirms transfer through the Time Event REST service and distinguishes malformed resources from resources stored for later functional validation.
<https://docs.oracle.com/en/cloud/saas/human-resources/faitl/how-you-import-data-from-time-collection-devices.html>

[03] Time Device Event Mappings - Defines mapping from supplier events to Oracle In, Out, In and Out, and Out and In application events.
<https://docs.oracle.com/en/cloud/saas/human-resources/faitl/time-device-event-mappings.html>

[04] Supplier-Related Choice Lists for Time Device Event Mappings - Defines Oracle supplier and supplier-event lookups, including ORA_HWM_TCD_SUPPLIERS and ORA_HWM_TCD_SUPPLIER_EVENTS.
<https://docs.oracle.com/en/cloud/saas/human-resources/faitl/supplier-related-choice-lists-for-time-device-event-mappings.html>

[05] Time Attributes for In and Out Event Matching - Explains how Oracle matches an Out event to the nearest prior In event and how time attributes affect matching.
<https://docs.oracle.com/en/cloud/saas/human-resources/faitl/time-attributes-for-in-and-out-event-matching.html>

[06] Manage In and Out Time Events - Provides end-to-end examples, request/response payloads and returned timeEventRequestId/timeEventId values.
https://docs.oracle.com/en/cloud/saas/human-resources/farws/Managing_In_and_Out_Time_Events.html

[07] Time Event Requests REST Endpoints - Canonical REST resource and GET/POST paths for timeEventRequests.

<https://docs.oracle.com/en/cloud/saas/human-resources/farws/api-time-event-requests.html>

[08] Time Events REST Endpoints - Child timeEvents REST resource and paths.

<https://docs.oracle.com/en/cloud/saas/human-resources/farws/api-time-event-requests-time-events.html>

[09] Time Collection Device Setup and Maintenance Tasks - Oracle setup objects: mappings, processing profiles, rules and generation of time cards from time-device events.

<https://docs.oracle.com/en/cloud/saas/human-resources/faitl/time-collection-device-setup-and-maintenance.html>

[010] Overview of Implementing Time and Labor - Confirms third-party badge/biometric devices as supported time-reporting methods and Oracle ownership of downstream processing.

<https://docs.oracle.com/en/cloud/saas/human-resources/faitl/overview-of-implementing-time-and-labor.html>

A. Appendix - current Oracle payload shape

```
{
  "requestNumber": "BSS-J35-01H...",
  "sourceId": "HWM_CLOCK_TIME",
  "requestTimestamp": "2026-10-09T16:45:12.000+00:00",
  "timeEvents": [
    {
      "deviceId": "J35",
      "eventDateTime": "2026-10-09T18:45:10.321+02:00",
      "supplierDeviceEvent": "BSS_START_WORK",
      "reporterId": "84729",
      "reporterIdType": "BADGE"
    }
  ]
}
```

This is the current BSS adapter shape, not a final tenant contract. The exact tenant URL, authentication, sourceId, reporterIdType, supplier event codes and optional attributes must be validated with the Oracle implementation team before production delivery is enabled.

B. Appendix - first improvement priorities

- Improve the terminal UI for real touch-screen use: strong visual feedback, short interaction path, auto reset and industrial kiosk behavior.
- Improve the Admin UI without mixing it with terminal mode: terminals, actions, event inspection, delivery backlog and diagnostics.
- Add terminal/device health and last-seen visibility without creating T&A business state.
- Add controlled reconciliation tooling for UNKNOWN/REJECTED deliveries before real Oracle connectivity is enabled.
- Add RFID through a replaceable input adapter once the physical reader interface is known.
- Expand tests around state transitions, duplicate scans/debounce, event immutability, Oracle mapping and failure recovery.
- Keep the authentication boundary pending until the embedding/standalone product decision is made.

END OF WORKING BASELINE